National University of Computer and Emerging Sciences 0062
Islamabad Campus

Data Structures (CS2001)

Sessional-II Exam

Course Instructor(s):
Mr. Muhammad Owais Idrees, Mr. Shehryar Rashid
Section(s): All Computer Science Sections

Total Time (Hrs): 1
Total Marks: 70
Total Questions: 6

Date: Nov 4, 2024

Roll No _____ **Course Section** _____ **Student Signature** _____

Do not write below this line.

Attempt all the questions.

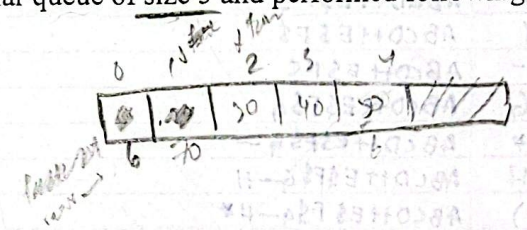
Instructions:

- ✓ **Read carefully!** If your brain asks a question, have a quiet little chat with it—**no need to disturb your neighbors** (they're thinking too!).
- ✓ **Stick to the space** provided! No need to wander off onto extra sheets. Everything you need is right here!
- ✓ **Answer every question on this paper.** If you think of any new questions, save them for later!
- ✓ **Please try to solve easy questions first, before attempting the tricky questions.**

Question 1 - Queues [4+4 = 8 marks]

Let's suppose you implemented the array-based circular queue of size 5 and performed following operation.

- Enqueue 10, 20, 30, 40, 50
- Dequeue
- Enqueue 6
- Dequeue
- Enqueue 70



1. What will be the values of front and rear?

   Value of Front: 30 Value of Rear: 70

2. What are the elements in the queue after these operations?

Index	0	1	2	3	4
Values	6	70	30	40	50

Question 2 - Stack [12 marks]

Part A: [8 Marks]
Precedence table:

Precedence	Operators	Associativity
1	++ (post increment), -- (pre decrement)	Left to right
2	$ (power, binary)	Right to left
3	*, / (binary)	Left to right
4	+, - (binary)	Left to right

Note: 1 has the highest precedence and 4 is the lowest precedence.

CamScanner

ABCD++EFG--H*L*+/+

Convert the infix string to post-fix string using stack:

Tricky part: "++" is post increment and "--" is predecrement operators

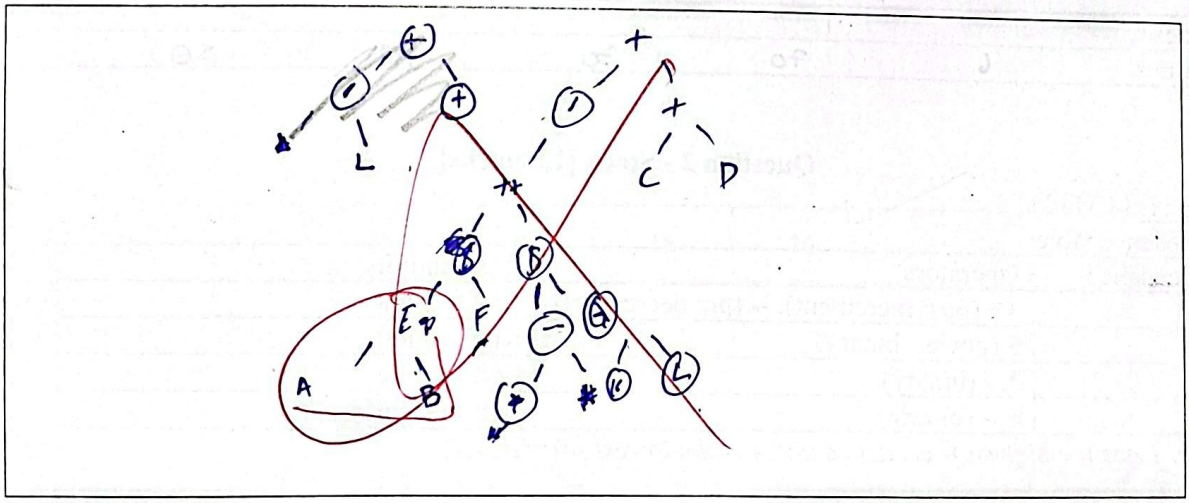
Note: if an error occurs in an earlier step, it will continue to affect subsequent steps.

Infix string "A + B / (C + D ++ $ E $ F / (-- G * H) * L)"

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
/	AB	+/
(	AB	+/(
C	ABC	+/(+
+	ABC	+/(+
D	ABCD	+/(+
++	ABCD	+/(+ ++
$	ABCD++	+/(+ $
E	ABCD++E	+/(+ $
$	ABCD++E$	+/(+ $
F	ABCD++E$F	+/(+ $
/	ABCD++EF	+/(+/
(	ABCD++EF	+/(+/(
--	ABCD++EF	+/(+/(--
G	ABCD++EFG	+/(+/(--
*	ABCD++EFG--	+/(+/(-- *
H	ABCD++EFG--H	+/(+/(-- *
)	ABCD++EFG--H*	+/(+/
*	ABCD++EFG--H*/	+/(+ *
L	ABCD++EFG--H*/L	
)	ABCD++EFG--H*/L*+/+	

Part B: [4 marks]

Now construct Expression tree:



CamScanner

Question 3 - BST [10 Marks]

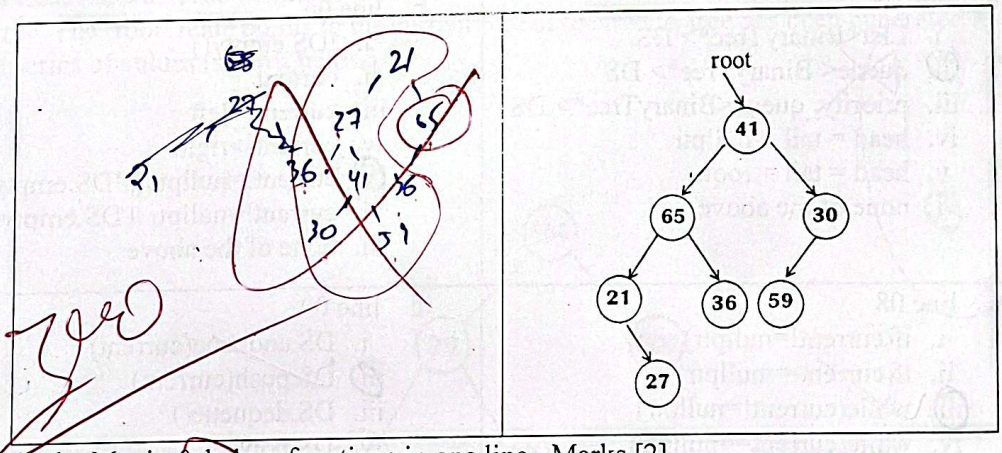
Given a code of Tree (Magic & helper function):

```
void helper(Tree * root, Tree *& first, Tree *& second, Tree *& third, Tree *& fourth) {
  if (root == nullptr)
    return;
  helper (root->left, first, second, third, fourth);
  if (fourth && root->data < fourth ->data) {
    if (!first) {
      first = fourth;
      second = root;
    }
    else
      third = root;
  }
  fourth = root;
  helper (root->right, first, second, third, fourth);
}
void Magic(Tree *root) {

  Node *first = nullptr, *second = nullptr, *third = nullptr, *fourth = nullptr;
  helper (root, first, second, third, fourth);

  if (first && third)
    swap(first->data, third ->data);
  else if (first && second)
    swap(first->data, second ->data);
}// ignore errors (if any)
```

1. Redraw the Tree after execution of the code (Magic function) [8 marks]



2. Explain the Magic & helper functions in one line. Marks [2]

traversing the tree

CamScanner

Question 4 - Tree Coding [1.5 * 10 = 15 Marks]

Consider the following (partially completed) code snippet for Converting Binary Tree to inorder Doubly Linked List (consider left as previous & right as next pointers) and answer the following questions.

```
01: void treeToDoublyLL(BinaryTree * root, BinaryTree *& head, BinaryTree *& tail)
02: {
03:     ____stack<BinaryTree*> DS____; //2 marks
04:     BinaryTree* current=root;
05:     bool flag=true;
06:     while(current!=nullptr)// 2 marks
07:     {
08:         _____{ // 2 marks
09:             _____; // 2 marks
10:             current=current->left;
11:         }
12:         _____; // 2 marks
13:     if(flag){
14:         _____; // 2 marks
15:         flag=false;
16:     }
17:     else{
18:         _____;// 2 marks
19:         _____;// 2 marks
20:         tail=current;
21:     }
22:     _____; // 2 marks
23:     _____; // 2 marks
24:     }
25: }
```

Encircle the correct options. *Note: any updates or alterations will result in zero marks*

a. line 03
   i. List<BinaryTree*> DS
   ii. queue< BinaryTree*> DS
   iii. priority_queue<BinaryTree*> DS
   iv. head = tail = nullptr
   v. head = tail = root;
   vi. none of the above

b. line 06
   i. !DS.empty()
   ii. current
   iii. current->left
   iv. current->right
   v. current!=nullptr || !DS.empty()
   vi. current!=nullptr || DS.empty()
   vii. none of the above

c. line 08
   i. if(current!=nullptr)
   ii. if(current==nullptr)
   iii. while(current!=nullptr)
   iv. while(current==nullptr)
   v. for(;current->left;)
   vi. none of the above

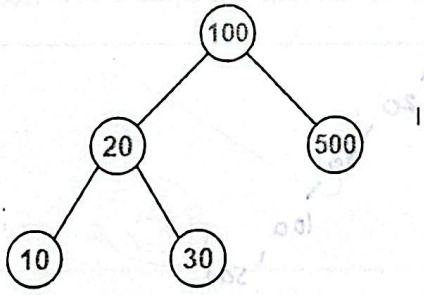
d. line 09
   i. DS.enqueue(current)
   ii. DS.push(current)
   iii. DS.dequeue()
   iv. DS.pop()
   v. DS.enqueue(current->left)
   vi. DS.push(current->left)
   vii. None of the above
   viii.

CamScanner

e. line 12 i. DS.enqueue(current) ii. DS.push(current) iii. DS.dequeue() iv. DS.pop() v. current=DS.front() vi. current=DS.top() vii. none of the above	f. line 14 i. tail=current ii. tail=current->left iii. tail=current->right iv. head=tail=current v. current=head=tail vi. none of the above
g. line 18 i. current=tail ii. current=DS.front() iii. current=DS.top() iv. tail->right=current v. tail->left=nullptr vi. root->left=tail vii. none of the above	h. line 19 i. current->left=tail ii. tail->right=current iii. current->right=tail iv. tail->left=nullptr v. root->left=tail vi. none of the above
i. line 22 i. DS.enqueue(current->right) ii. DS.push(current->right) iii. DS.pop() iv. DS.dequeue() v. current=DS.front() vi. current=DS.top() vii. none of the above	j. line 23 i. current=current->right ii. current=current->left iii. current=tails iv. no need of this statement (remove it) v. all of the above vi. none of the above

Question 5 - Tree Dry Run [10 Marks]

You are provided with a block of code and a tree structure. Assume these functions are part of a class named `Tree`, which contains methods for inserting, deleting, and updating nodes in the tree. The `root` node points to the current root of the tree. A tree has been generated by inserting a series of values (shown below).



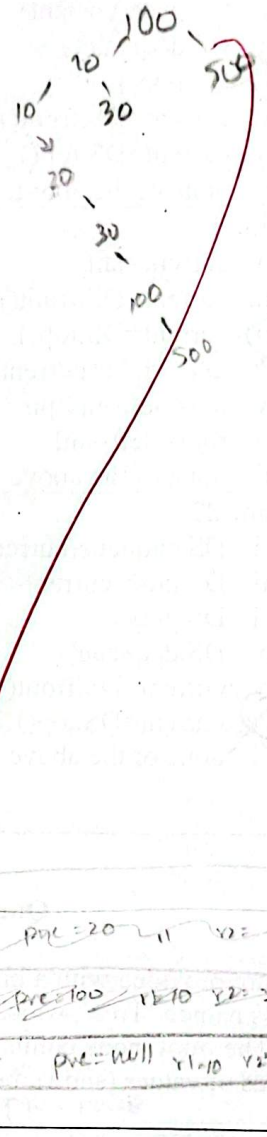
CamScanner

curr = 100
Pre = null

```
node* UpdateTree(node* curr, node* pre) {
    count++;
    if (curr->left == NULL && curr->right == NULL)
        return curr;
    node* r1 = NULL;
    node* r2 = NULL;
    if (curr->left != NULL)
        r1 = UpdateTree(curr->left, curr);
    if (curr->right != NULL)
        r2 = UpdateTree(curr->right, curr);
    if (r1 == NULL)
        return curr;
    if (r1->left != NULL)
    {
        r1->left->right = r1;
        r1 = r1->left->right;
        r1->left = NULL;
    }
    node* tmp = r1;
    while (tmp->right != NULL)
        tmp = tmp->right;
    tmp->right = curr;
    curr->right = r2;
    curr->left = NULL;
    return r1;
}

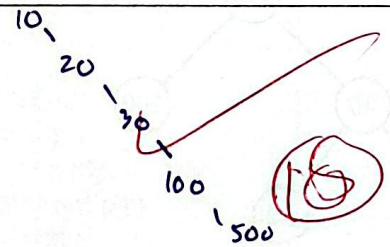
void UpdateTree() {
    if (root != NULL) {
        node* pre = NULL;
        root = UpdateTree(root, pre);
    }
}
```

Rough Work



count = 1

What will be the resultant tree if we call function **Update Tree? (Final Tree Only)**



CamScanner

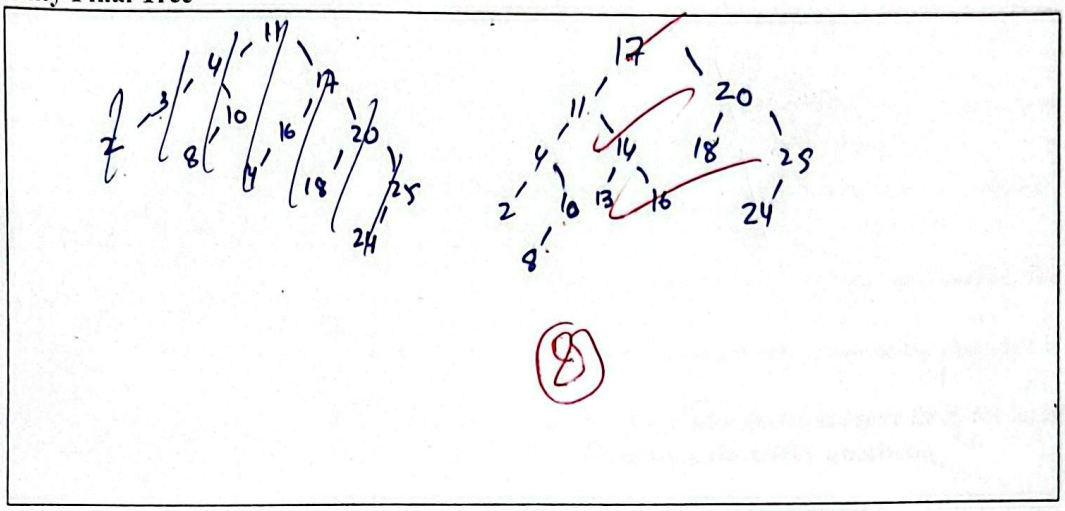
Question 6 – AVL Tree [8 + 7 = 15 Marks]

You are given with following values.

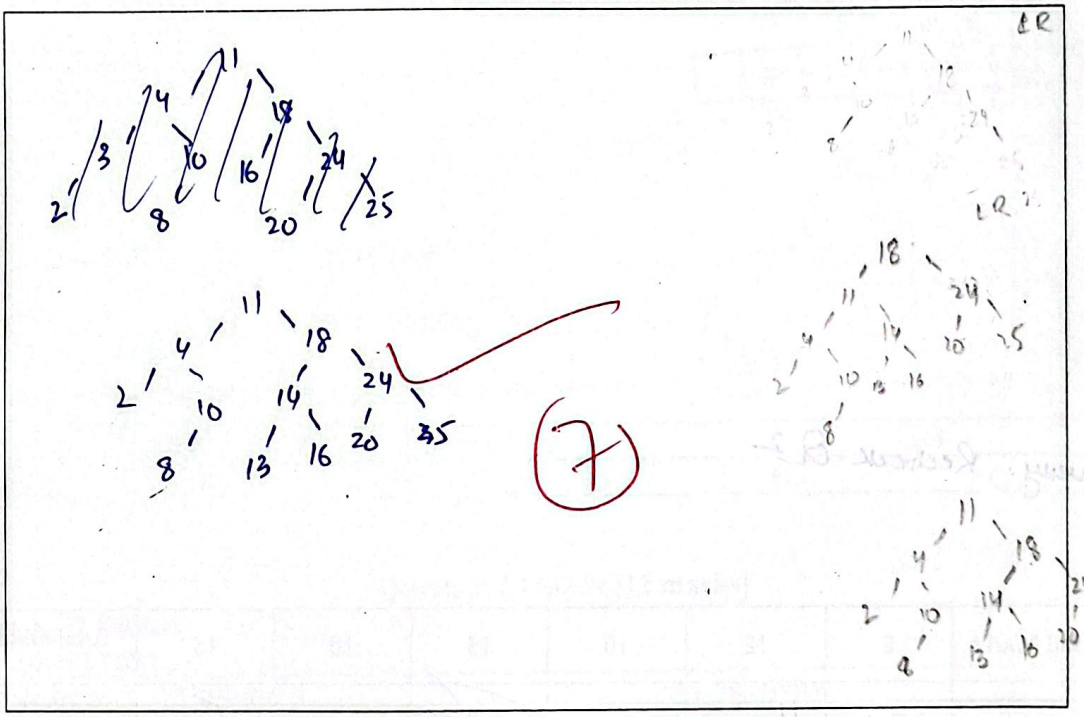
17, 11, 20, 4, 13, 25, 2, 10, 16, 14, 18, 24, 8

1. Draw the resultant tree after inserting the all above values.

Only Final Tree



2. Delete 17 using in order successor, from above constructed tree, then draw the resultant tree.



CamScanner